

RankMixer：工业推荐系统中的排序模型扩展

Jie Zhu*, Zhifang Fan*, Xiaoxie Zhu*, Yuchen Jiang*, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, Huizhi Yang, Zheng Chai, Zhe Chen, Yuchao Zheng, Qiwei Chen[†], Feng Zhang, Xun Zhou, Peng Xu, Xiao Yang, Di Wu, Zuotao Liu

ByteDance

{zhujie.zj, zhuxiaoxie.777, jiangyuchen.jyc, wanghangyu.123, hanxintian, dinghaoran, xinmin.wang, zhaowenlin, gongzhen.666, yanghuizhi, chaizheng.cz, chenzhe.john, zhengyuchao.yc, feng.zhang, zhouxun, xupeng, wuqi.shaw, di.wu, michael.liu}@bytedance.com, {fanzhifangzf, chenqiwei05}@gmail.com

摘要

近年来,大语言模型(LLM)的进展激发了人们对扩展推荐系统的兴趣,但仍面临两个实际障碍。第一,工业级推荐系统的训练与服务成本必须满足严格的延迟上限和高 QPS 需求。第二,排序模型中大多数由人工设计的特征交叉模块沿袭自 CPU 时代,无法充分利用现代 GPU,导致模型浮点运算利用率(MFU)较低且可扩展性不佳。我们提出 RankMixer,这是一种面向统一、可扩展特征交互架构而定制的硬件感知模型设计。RankMixer 保留了 Transformer 的高并行性,同时以多头 token 混合模块取代二次复杂度的自注意力,从而获得更高效率。此外,RankMixer 通过逐 token FFN,同时保留了对不同特征子空间的建模能力以及跨特征空间的交互能力。为获得更高的 ROI,我们进一步通过 Sparse-MoE 变体将其扩展到十亿参数规模。我们采用动态路由策略,以解决专家训练不充分及不均衡的问题。实验表明,RankMixer

在万亿规模的生产数据集上展现出卓越的扩展能力。通过使用 RankMixer 替换之前多种多样、手工设计且 MFU 较低的模块,我们将模型 MFU 从 **4.5%** 提升至 **45%**,并在推理延迟大致不变的情况下,将在线排序模型的参数规模扩大了两个数量级。我们通过两个核心应用场景(推荐和广告)中的在线 A/B 测试,验证了 RankMixer 的通用性。最后,我们在不增加服务成本的情况下,上线了具有 1B 稠密参数的 RankMixer 以服务全部流量,使用户活跃天数提升 **0.3%**,应用内总使用时长提升 **1.08%**。

CCS Concepts

• 信息系统 → 推荐系统.

Keywords

缩放定律, 排序模型, 推荐系统

* 同等贡献。

† 通讯作者。

1 引言

推荐系统 (Recommender System, RS) 在信息分发过程中至关重要。作为一个重要的机器学习场景, RS 基于大量多域特征数据预测用户对物品的行为, 这些数据包括各类统计量等数值特征、用户和物品 ID 等类别特征、用户行为特征以及内容特征 [19, 41]。当前先进的推荐方法基于深度学习推荐模型 (Deep Learning Recommendation Models, DLRMs), 其在特征输入嵌入之上使用神经网络作为稠密交互层, 灵活地捕获特征交互。DLRM 中的稠密交互层对 RS 性能至关重要, 为此人们提出了多种模型结构 [7, 11, 33, 38, 42]。

受益于参数量增长的大语言模型 (Large Language Models, LLMs) 不断取得进展 [1, 14, 16], 扩大 DLRM 规模以充分利用海量数据已成为迫切需求。以往研究在 DLRM 规模扩展方面取得了许多成果: 早期研究 [2, 6, 40] 仅加宽或堆叠特征交互层, 而不修改结构。这种方式带来的收益有限, 有时甚至为负 [18, 32]。随后, DHEN [39] 和 Wukong [38] 等工作专注于设计创新的 DNN 结构, 以提升规模扩展性能。然而, 在推荐场景中利用模型规模提升性能面临独特的实际挑战。与 NLP 或视觉任务不同, 工业级推荐系统必须严格满足紧迫的延迟约束, 并支持极高的 QPS (每秒查询数)。因此, 核心挑战在于找到兼顾模型效果与计算效率的最佳平衡点。

长期以来, 推荐排序模型的架构一直受 CPU 时代设计原则影响。这类模型通常组合异构、多样的手工交叉特征模块来提取特征交互, 但在现代 GPU 上, 其许多核心算子受内存带宽而非计算能力限制, 导致 GPU 并行度较差, MFU (Model FLOPs Utilization, 模型 FLOPs 利用率) 极低, 通常只有个位数百分比。此外, 由于 CPU 时代模型的计算成本大致与参数量成正比, 缩放定律所暗示的激进规模扩展的潜在 ROI 在实践中难以实现。

综上, DLRM 缩放定律研究必须克服以下问题:

- 架构应与硬件对齐, 在现代 GPU 上最大化 MFU 和计算吞吐量。
- 模型设计必须利用推荐数据的特性, 例如异构特征空间以及数百个域之间个性化的交叉特征交互。

为应对这些挑战, 我们提出了一种硬件感知的模型设计方法 RankMixer。RankMixer 的核心设计基于两个可扩展组件: 1. 多头 *token* 混合仅通过无参数算子实现跨 *token* 特征交互, 在性能和计算效率方面均优于自注意力机制。2. 逐 *token* 前馈网络 (FFN) 通过为不同特征子空间建模分配独立参数, 显著扩展模型容量, 并解决特征空间间的主导问题。这些 FFN 也与推荐数据模式高度契合, 从而展现出更好的规模扩展行为。为进一步提升大规模模型的 ROI, 我们将逐 *token* FFN 模块扩展为稀疏混合专家 (MoE) 结构。通过针对不同数据为每个 *token* 动态地仅激活一部分专家, 我们能够在计算成本仅略微增加的情况下显著提升模型容量。RankMixer 采用类似 Transformer 的高度并行架构, 但克服了基于自注意力的特征交互的若干关键局限: 训练效率低、建模跨空间 ID 相似性时出现组合爆炸, 以及注意力权重矩阵导致严重的内存瓶颈。同时, 与原生 Transformer 相比, 在相同 FLOPs 下 RankMixer 具有更大的模型容量和学习能力。

在抖音推荐系统的生产部署中, 我们证明了, 相较此前的基线, 可以在保持更短推理延迟的同时将模型参数扩大 100× 以上。这得益于 RankMixer 架构能够将参数增长与 FLOPs 解耦, 并通过高 MFU 和工程优化将 FLOPs 增长与实际成本解耦。

主要贡献可概括如下:

- 我们提出了一种遵循硬件感知模型设计理念的新架构 RankMixer。我们设计了多头 *token* 混合和逐 *token* FFN 策略, 以高效捕获异构特征交互, 并利用动态路由策略提高 RankMixer 中 SparseMoE 的可扩展性。
- 借助高 MFU 和性能优化等手段, 包括提升 MFU 和量化, 我们在不增加推理成本的情况下将模型参数扩大了 70×。
- 我们开展了广泛的离线与在线实验, 并在万亿规模工业推荐数据集上研究模型的缩放定律。RankMixer 模型已成功部署到抖音 Feed 推荐排序并提供全流量服务, 使活跃天数和 App 使用时长分别提升了 0.3% 和 1.08%。

2 相关工作

现代推荐系统基于深度学习推荐模型 (DLRM), 如何有效建模特征交互是 DLRM 的关键因素 [5, 11, 15, 22, 31, 33, 39]。Wide&Deep [5] 是最早的尝试之一, 它结合了逻辑回归 (宽部分) 和 DNN (深部分), 分别捕捉低阶和高阶特征交互。DeepFM [11] 是另一项成果, 它集成了因子分解机 (FM) 和 DNN。此外, DeepCross [26] 是残差网络 [13] 的扩展, 旨在隐式地自动学习特征交互。然而, 仅依靠 DNN 学习高阶特征交互已被证明极具挑战性 [22, 25]。显式交叉方法设计不同的算子来显式捕捉高阶特征交互, 例如 PNN [22]、DCN [32] 及其后继者 DCNv2 [33]、xDeepFM [18]、FGCNN [20] 和 FiGNN [17]。AutoInt [28] 和 Hi-former [10] 使用带残差连接的注意力机制来学习复杂交互。DHEN [39] 提出将多种交互算子组合在一起。尽管提高了准确率, 这些新架构也增加了模型延迟和内存消耗, 且其模型规模相对较小。

缩放定律已成为深度学习的一个基础主题, 也是过去十年间促成诸多突破的关键催化剂, 尤其体现在自然语言处理 (NLP) [14, 16]、计算机视觉 (CV) [8, 37] 和多模态建模 [23, 24] 领域。它描述模型性能与模型规模、数据量和计算能力等缩放因子之间的幂律关系 [1, 16, 29, 30]。近年来, 推荐系统中的缩放定律引起了研究人员的广泛关注 [12]。相关研究已探索用户活动序列预训练 [6]、通用用户表示 [27, 40] 和在线检索 [9, 34] 的缩放策略。Wukong [38] 通过堆叠 FM 和 LCB 来学习特征交互。与此正交地, Zhang et al. [40] 将序列推荐模型扩展到了 0.8B 参数。HSTU [36] 增强了生成式推荐器 (GR) 的缩放效果, 后者更关注序列部分。

3 方法

3.1 整体架构

RankMixer 的整体架构由 T 个输入 token 组成, 它们经过 L 个连续的 RankMixer 块处理, 随后进入输出池化算子。如图 1 所示, 每个 RankMixer 块包含两个主要组件: (1) 多头 Token 混合层; (2) 逐 Token 前

馈网络 (PFFN) 层。首先, 输入向量 $\mathbf{e}_{\text{input}}$ 被 token 化为 T 个特征 token $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$, 每个 token 都表示一个语义一致的特征向量。RankMixer 块通过以下过程, 在 L 层中迭代细化 token 表示:

$$\begin{aligned}\mathbf{S}_{n-1} &= \text{LN}(\text{TokenMixing}(\mathbf{X}_{n-1}) + \mathbf{X}_{n-1}), \\ \mathbf{X}_n &= \text{LN}(\text{PFFN}(\mathbf{S}_{n-1}) + \mathbf{S}_{n-1}),\end{aligned}\quad (1)$$

其中, $\text{LN}(\cdot)$ 是层归一化函数, $\text{TokenMixing}(\cdot)$ 和 $\text{PFFN}(\cdot)$ 分别是多头 Token 混合模块和逐 token FFN 模块, $\mathbf{X}_n \in \mathbb{R}^{T \times D}$ 是第 n 个 RankMixer 块的输出, $\mathbf{X}_0 \in \mathbb{R}^{T \times D}$ 由 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ 堆叠而成, D 是模型的隐藏维度。输出表示 $\mathbf{o}_{\text{output}}$ 由最后一层的表示 $\mathbf{X}_L$ 经均值池化得到, 并将用于计算不同任务的预测。

3.2 输入层与特征 Token 化

构建大规模推荐模型的第一步是准备信息丰富的输入, 例如: **用户特征**包括用户 ID 和其他用户信息; **候选项特征**包括视频 ID、作者 ID 等; **序列特征**经序列模块 [4, 42] 处理以捕获时序兴趣, 得到 $\mathbf{e}_s$; 以及 **交叉特征**, 即用户与候选项之间的交叉特征。所有特征都将转换为不同维度的嵌入。

为了在后续阶段实现高效并行计算, 必须将不同维度的嵌入转换为维度对齐的向量, 即特征 token。我们将这一嵌入对齐过程称为 token 化。最简单的策略是为每个特征分配一个嵌入, 但鉴于特征通常多达数百个, 这会带来若干挑战。数百个 token 不可避免地会将分配给每个 token 的参数和计算量压缩到很小, 导致重要特征建模不充分, 且 GPU 核心利用不足。相反, token 过少 (例如只有一个 token) 会使模型结构退化为简单的深度神经网络 (DNN), 无法明确表示多样的特征空间, 从而带来主导特征遮蔽其他特征的风险。

为解决这些问题, 我们提出一种融合领域知识的基于语义的 token 化方法, 将特征分组为若干语义一致的簇。这些分组特征依次拼接为一个嵌入向量 $\mathbf{e}_{\text{input}} = [\mathbf{e}_1; \mathbf{e}_2; \dots; \mathbf{e}_N]$, 随后被划分为数量适当、维度大小固定的 token。每个特征 token $x_i \in \mathbb{R}^D$ 都包含一组表

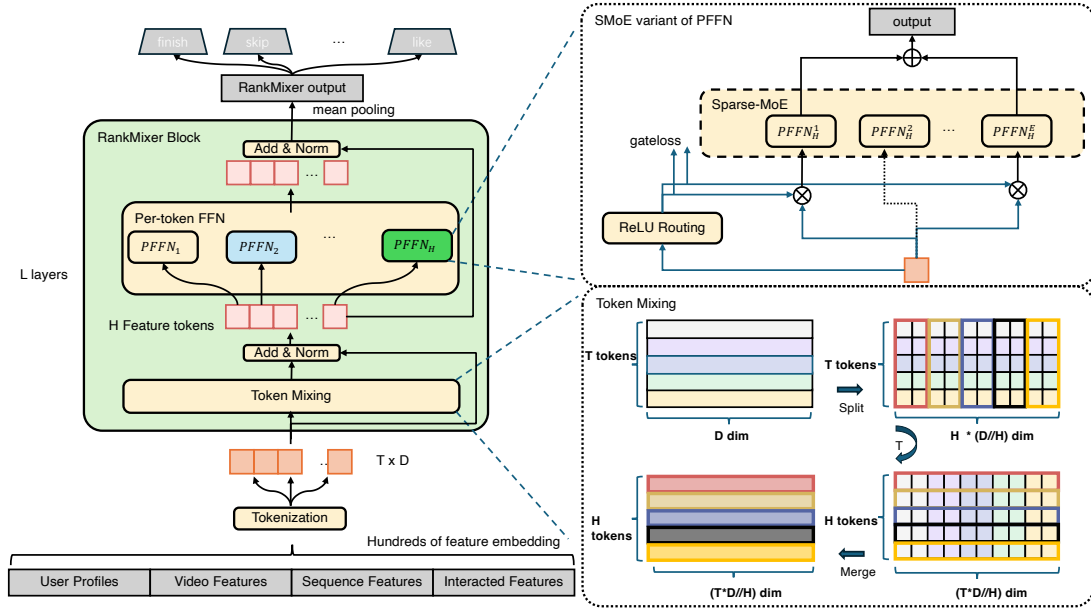


图 1: RankMixer 块的架构。一个 RankMixer 块由两个模块组成：多头 Token 混合和 基于 SMOE 的逐 token FFN。token 混合模块将每个 token 的嵌入划分为 H 个较小的部分（头），然后在 token 之间重新组合这些部分，以创建新的混合 token。这使得来自不同特征的信息能够相互交互。

示相似语义方面的特征嵌入。

$$\mathbf{x}_i = \text{Proj}(e_{\text{input}}[d \cdot (i-1) : d \cdot i]), \quad i = 1, \dots, T, \quad (2)$$

其中, e_{input} 为拼接后的嵌入向量, d 为每个 token 的固定维度, N 为特征组数量, T 为最终得到的 token 数量, 而 Proj 函数将拆分后的嵌入映射到 D 维。

3.3 RankMixer 块

3.3.1 多头 Token 混合. 为了促进 token 之间的有效信息交换（这对特征交叉和全局信息建模至关重要），我们引入了多头 Token 混合模块。每个 token 被均匀划分为 H 个头，token $\mathbf{x}_t$ 的第 h 个头记为 $\mathbf{x}_t^h$ ：

$$\left[\mathbf{x}_t^{(1)} \parallel \mathbf{x}_t^{(2)} \parallel \dots \parallel \mathbf{x}_t^{(H)} \right] = \text{SplitHead}(\mathbf{x}_t). \quad (3)$$

由于推荐任务需要综合考虑不同视角，这些头可以视为将 token $\mathbf{x}_t$ 投影到低维特征子空间。Token 混合用于融合这些子空间向量，以实现全局特征交互。形式化地，多头 Token 混合后与第 h 个头对应的第 h 个 token $\mathbf{s}^h$ 构造如下：

$$\mathbf{s}^h = \text{Concat}(\mathbf{x}_1^h, \mathbf{x}_2^h, \dots, \mathbf{x}_T^h). \quad (4)$$

多头 Token 混合模块的输出为 $\mathbf{S} \in \mathbb{R}^{H \times \frac{TD}{H}}$ ，由所有重排后的 token $\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_H$ 堆叠而成。在本工作中，我们设置 $H = T$ ，以便在 Token 混合后保持 token 数量不变，从而进行残差连接。

经过残差连接和归一化模块后，我们可以得到

$$\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_T = \text{LN}(\text{TokenMixing}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T) + (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)), \quad (5)$$

尽管自注意力在大语言模型中已被证明非常有效, 但我们发现它并不适合推荐系统。在自注意力中, 注意力权重通过 token 的内积计算。这种方法对 NLP 很有效, 因为所有 token 共享统一的嵌入空间。然而, 在推荐任务中, 特征空间在本质上是异质的。计算两个异质语义空间之间的内积相似度是公认的难题——尤其是在推荐系统中, 来自用户侧和物品侧的特征 ID 空间可能包含数亿个元素。因此, 将自注意力应用于如此多样的输入, 其性能并不优于无参数的多头 Token 混合方法, 却会消耗更多计算量、内存 IO 操作和 GPU 显存。

3.3.2 逐 token FFN. 以往的 DLRM 和 DHEN 模型往往在单个交互模块中混合来自许多不同语义空间的特征, 这可能导致高频字段占据主导地位, 淹没低频或长尾信号, 并最终损害整体推荐质量。我们引入了一种参数隔离的前馈网络架构, 称为逐 token FFN。在传统设计中, FFN 的参数在所有 token 之间共享; 而我们的方法使用专用变换处理每个 token, 从而隔离每个 token 的参数。对于第 t 个 token $\mathbf{s}_t$, 逐 token FFN 可表示为

$$\mathbf{v}_t = f_{\text{pffn}}^{t,2} \left(\text{Gelu} \left(f_{\text{pffn}}^{t,1} (\mathbf{s}_t) \right) \right), \quad (6)$$

其中

$$f_{\text{pffn}}^{t,i}(\mathbf{x}) = \mathbf{x} \mathbf{W}_{\text{pffn}}^{t,i} + \mathbf{b}_{\text{pffn}}^{t,i} \quad (7)$$

是逐 token FFN 的第 i 层 MLP, $\mathbf{W}_{\text{pffn}}^{t,1} \in \mathbb{R}^{D \times kD}$, $\mathbf{b}_{\text{pffn}}^{t,1} \in \mathbb{R}^{kD}$, $\mathbf{W}_{\text{pffn}}^{t,2} \in \mathbb{R}^{kD \times D}$, $\mathbf{b}_{\text{pffn}}^{t,2} \in \mathbb{R}^D$, k 是用于调节逐 token FFN 隐藏维度的超参数, $\text{Gelu}(\cdot)$ 是 Gelu 激活函数, $\mathbf{s}_t \in \mathbb{R}^D$ 是第 t 个 token。

我们将逐 token FFN 模块总结为

$$\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_T = \text{PFFN}(\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_T), \quad (8)$$

其中

$$\text{PFFN}(\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_T) = f_{\text{pffn}}^{t,2} \left(\text{Gelu} \left(f_{\text{pffn}}^{t,1} (\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_T) \right) \right). \quad (9)$$

与参数全部共享的 FFN 相比, 逐 token FFN 通过引入更多参数增强了建模能力, 同时保持计算复杂度不变。

需要强调的是, 逐 token FFN 与 MMoE 专家不同: 每个逐 token FFN 接收不同的 token 输入, 而 MMoE 中的所有专家共享相同的输入。不同于多个专家处理同一输入的 MMoE, 也不同于不同输入共享一个 FFN 的 Transformer, RankMixer 同时拆分输入和参数, 这有利于在不同特征子空间中学习多样性。

3.4 RankMixer 中的稀疏 MoE

为进一步提高扩展 ROI, 我们可以将每个逐 token 稠密 FFN 替换为 稀疏混合专家 (MoE) 块, 从而在计算成本大致不变的同时增大模型容量。然而, 原始 Sparse-MoE 在 RankMixer 中会出现性能下降, 原因如下: (i) **统一的 k -专家路由**。Top- k 选择对所有特征 token 一视同仁, 会在低信息量 token 上浪费预算, 而使高信息量 token 得不到充分资源, 阻碍模型捕捉 token 之间的差异。(ii) **专家训练不足**。逐 token FFN 已经将参数量乘以 $\#\text{tokens}$; 再添加不共享专家会进一步导致专家数量激增, 从而造成路由高度不均衡以及专家训练不充分;

我们结合两种互补的训练策略来解决上述问题。

ReLU 路由. 为赋予 token 灵活的专家数量并保持可微性, 我们用 ReLU 门控加自适应 ℓ_1 惩罚 [35] 取代常见的 Topk+softmax。对于 token $\mathbf{s}_i \in \mathbb{R}^{d_h}$ 的第 j 个专家 $e_{i,j}(\cdot)$ 和路由器 $h(\cdot)$:

$$G_{i,j} = \text{ReLU}(h(\mathbf{s}_i)), \quad \mathbf{v}_i = \sum_{j=1}^{N_e} G_{i,j} e_{i,j}(\mathbf{s}_i), \quad (10)$$

其中, N_e 表示每个 token 的专家数, N_t 表示 token 数。ReLU 路由会为信息量高的 token 激活更多专家, 从而提高参数效率。稀疏性由 $\mathcal{L}_{\text{reg}}$ 及其系数 λ 调控, 以使平均活跃专家比例保持在预算附近:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda \mathcal{L}_{\text{reg}}, \quad \mathcal{L}_{\text{reg}} = \sum_{i=1}^{N_t} \sum_{j=1}^{N_e} G_{i,j}. \quad (11)$$

稠密训练 / 稀疏推理 (*DTSI-MoE*)。受 [21] 启发, 我们采用两个路由器 h_{train} 和 h_{infer} , 且 $\mathcal{L}_{\text{reg}}$ 仅应用于 h_{infer} 。训练期间会同时更新 h_{train} 和 h_{infer} , 而

推理时仅使用 h_{infer} 。结果表明, DS-MoE 能在降低推理成本的同时避免专家训练不足。

3.5 扩展方向

RankMixer 本质上是一种高度并行且可扩展的架构。其参数量和计算成本可沿四个正交轴扩展: Token 数量 T 、模型宽度 D 、层数 L 和专家数量 E 。对于全稠密激活版本, 单个样本的参数量和前向 FLOPs 可计算为

$$\# \text{Param} \approx 2kLT D^2, \quad \text{FLOPs} \approx 4kLT D^2, \quad (12)$$

k 是用于调整 FFN 隐藏维度的缩放比例。在 *Sparse-MoE* 版本中, 每个 token 的有效参数量和计算量还会按稀疏比率 $s = \frac{\# \text{Activated_Param}}{\# \text{Total_Param}}$ 进一步缩放。

4 实验

4.1 实验设置

4.1.1 数据集与环境. 离线实验使用抖音推荐系统的训练数据进行。这些数据来自抖音的在线日志和用户反馈标签。训练数据集包含 300 多个特征, 例如数值特征、ID 特征、交叉特征和序列特征, 涉及数十亿用户 ID 和数亿视频 ID, 它们均被转换为嵌入。数据每日覆盖数万亿条记录, 实验在两周期间收集的数据上进行。

4.1.2 评估指标. 为评估模型性能, 我们使用 AUC(曲线下面积) 和 UAUC(用户级 AUC) 作为主要性能指标, 并使用参数量、FLOPs 和 MFU 作为效率指标, 具体如下: **完播/跳过 AUC/UAUC:** 完播 =1/0 或跳过 =1/0 标签表示用户是否看完某个视频, 或是否在短时间内滑到下一个视频。我们评估该完播标签的 AUC 和 UAUC。AUC 提升 0.0001 即可视为具有可信的显著性。 **Dense-Param:** 稠密部分的参数量, 不包括稀疏嵌入参数。训练 **FLOPs/Batch:** 单个大小为 512 的批次通过模型所需的浮点运算次数 (FLOPs), 表示训练的计算成本。 **MFU:** MFU (模型 FLOPs 利用率) 是衡量模型对硬件所提供浮点运算能力利用效

表 1: 约 1 亿参数推荐模型的性能与效率对比 (最优值以粗体标出)。

模型	完播		跳过		效率	
	AUC↑	UAUC↑	AUC↑	UAUC↑	参数量	FLOPs/批次
DLRM-MLP (base)	0.8554	0.8270	0.8124	0.7294	8.7 M	52 G
DLRM-MLP-100M	+0.15%	—	+0.15%	—	95 M	185 G
DCNv2	+0.13%	+0.13%	+0.15%	+0.26%	22 M	170 G
RDCN	+0.09%	+0.12%	+0.10%	+0.22%	22.6 M	172 G
MoE	+0.09%	+0.12%	+0.08%	+0.21%	47.6 M	158 G
AutoInt	+0.10%	+0.14%	+0.12%	+0.23%	19.2 M	307 G
DHEN	+0.18%	+0.26%	+0.36%	+0.52%	22 M	158 G
HiFormer	+0.48%	—	—	—	116 M	326 G
Wukong	+0.29%	+0.29%	+0.49%	+0.65%	122 M	442 G
RankMixer-100M	+0.64%	+0.72%	+0.86%	+1.33%	107 M	233 G
RankMixer-1B	+0.95%	+1.22%	+1.25%	+1.82%	1.1 B	2.1T

率的指标, 计算方式为模型实际 FLOPs 消耗除以硬件理论 FLOPs 容量。

4.1.3 基线. 我们与以下广受认可的 SOTA 基线进行比较: **DLRM-MLP:** 作为实验基线的原始 MLP, 用于特征交叉; **DCNv2**[33]、**RDCN**[3], 即特征交叉模型的 SOTA。 **MoE** 通过使用多个并行专家进行扩展。 **AutoInt**[28]、**Hiformer**[10] 结合了异构自注意力层与低秩近似矩阵计算。 **DHEN**[39]: 结合不同类型的特征交叉块, 并堆叠多层 (DCN/自注意力/FM/LR)。 **Wukong**[38] 沿用 DHEN 的架构, 通过因子分解机块 (FMB) 和线性压缩块 (LCB) 研究特征交互的缩放定律。

所有实验均在数百块 GPU 上通过混合分布式训练框架进行, 其中稀疏部分异步更新, 而稠密部分同步更新。所有模型的优化器超参数均保持一致。稠密部分使用学习率为 0.01 的 RMSProp 优化器, 稀疏部分则使用 Adagrad 优化器。

4.2 与 SOTA 方法的比较

为了探索如何扩大模型规模, 我们比较了参数量均在约 1 亿、规模相近的模型, 以确定在相同计算成本下表现最佳的模型结构。

我们的方法与各基线的主要性能结果汇总于表 1。可以看到, RankMixer 在多个目标和指标上均显著优于其他 SOTA 模型。

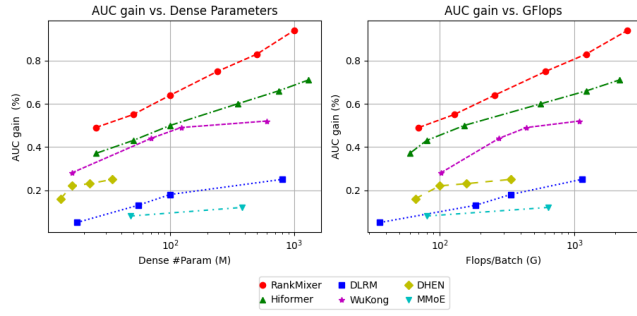


图 2: 不同模型的最终 AUC 增益与参数量/FLOPs 之间的缩放定律。x 轴采用对数刻度。

我们进一步仔细分析各个模型。首先, 仅将 DLRM 扩展到 1 亿参数只能带来有限收益, 这凸显了针对推荐数据特性设计模型对于获得更好规模扩展性能的必要性。随后, 我们将 RankMixer 与 DCN、RDCN、AutoInt 和 DHEN 等经典交叉结构设计进行比较, 发现这些模型存在参数量与计算成本失衡的问题。即便参数量相对较小, 这些模型的 FLOPs 也已很高, 说明其设计缺陷限制了表中结果。与此同时, 尽管 RankMixer 取得了最佳性能, 但扩展到 1 亿参数时, 其 FLOPs 在所有模型中仍处于相对适中的水平, 体现了模型容量与计算负载之间的平衡。

我们还将 RankMixer 与 Hiformer、Wukong 等若干常用的先进规模扩展模型进行比较, 结果发现, 在参数设置相近的情况下, RankMixer 不仅性能更好, 计算需求也更低。

4.3 不同模型的缩放定律

在图 2 中, 我们展示了从参数规模和 FLOPs 两个角度观察到的缩放定律曲线。无论就参数还是 FLOPs 而言, RankMixer 模型都展现出最陡峭的缩放定律曲线。RankMixer 始终优于其他模型。

尽管 wukong 模型的参数曲线相对陡峭, 但其计算成本增长得更快; 因此, 在 AUC 与 FLOPs 曲线上, 它与 RankMixer 和 hiformer 的差距更大。此外, hiformer 的性能略逊于 RankMixer, 这反映出它对特征层面的 token 分割和 Attention 的依赖会影响效率。DHEN 的扩展表现不理想, 反映出其交叉结构的可扩展

展性有限。另外, MoE 通过增加专家来扩展的策略会带来保持专家均衡的挑战, 从而导致扩展性能不佳。

具体而言, 我们可以通过增大宽度 (D)、特征 token 数 (T) 和层数 (L) 来扩展 RankMixer 模型。在实验中, 我们观察到一个与 LLM 缩放定律一致的结论: 模型质量主要与参数总量相关, 而不同的扩展方向 (深度 L 、宽度 D 、token 数 T) 会带来几乎相同的性能。从计算效率的角度看, 较大的隐藏维度会产生更大的矩阵乘形状, 因而比堆叠更多层获得更高的 MFU。因此, 100M 和 1B 模型的最终配置分别设为 ($D=768$, $T=16$, $L=2$) 和 ($D=1536$, $T=32$, $L=2$)。

4.4 消融实验

表 2: RankMixer-100M 组件消融实验

设置	Δ AUC
无跳跃连接	-0.07%
无多头 token 混合	-0.50%
无层归一化	-0.05%
逐 token FFN $\rightarrow$ 共享 FFN	-0.31%

表 3: Token2FFN 路由策略比较

路由策略	Δ AUC	Δ 参数量	Δ FLOPs
All-Concat-MLP	-0.18%	0.0%	0.0%
All-Share	-0.25%	0.0%	0.0%
Self-Attention	-0.03%	+16%	+71.8%

在 RankMixer-100M 模型中, 我们对残差连接和多头 Token-Mixing 进行了消融实验。从表 2 可以看出, 移除这些组件会显著降低模型性能。移除多头 Token-Mixing 会丢失全局信息, 因为每个 FFN 只对部分特征建模, 彼此之间没有交互。移除残差连接和 LayerNorm 也会导致性能下降, 降低训练稳定性, 并更容易出现梯度爆炸或消失问题。

我们进一步分析了 token 混合策略, 即表 3 中从特征 token 到 FFN 的路由策略。与多头 Token-Mixing (Multi-Head Token-Mixing) 比较的路由策略

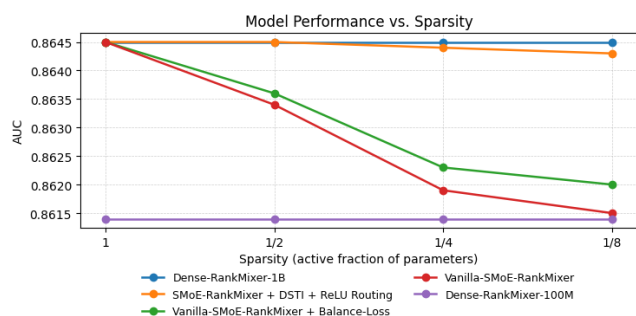


图 3: 随稀疏激活比例逐步降低 (专家的 1、1/2、1/4、1/8) 时 RankMixer 变体的 AUC 性能: 稠密训练 + ReLU 路由的 SMoE 几乎保留了 1 B 稠密模型的全部准确率。

包括: *All-Concat-MLP*: 将所有 token 拼接后通过一个大型 MLP 处理, 然后拆分成数量相同的 token。性能下降表明, 学习大矩阵存在挑战, 同时还会削弱局部信息学习。*All-Share*: 不进行拆分, 整个输入向量被共享, 并像 MoE 一样送入每个逐 token FFN。性能显著下降, 这表明与全共享输入相比, 特征子空间拆分和独立建模非常重要。*Self-Attention*: 在 token 之间应用自注意力进行路由。其性能略逊于多头 Token-Mixing, 且计算成本很高, 这表明要学习数百个不同特征子空间之间的相似性并不容易。

4.5 稀疏 MoE 的可扩展性与专家均衡

可扩展性。图 3 展示了离线 AUC 增益随 SMoE 稀疏度的变化。将稠密训练—稀疏推理与 ReLU 路由结合, 是在高稀疏度下保持精度的关键; 它使 RankMixer 能够将参数容量 (及内存占用) 扩展超过 8 $\times$, 同时几乎不损失 AUC, 并显著节省推理时间 (吞吐量提升 +50%)。随着激活专家数量减少, 原生 SMoE 的性能单调下降, 这说明了我们所指出的专家不均衡与训练不足问题。加入负载均衡损失后, 相比原生 SMoE, 性能退化有所缓解, 但仍不及 DTISI + ReLU 版本, 因为问题主要出在专家训练而非路由器上。这验证了 Sparse-MoE 是将 RankMixer 从当前 1 B 参数扩展到

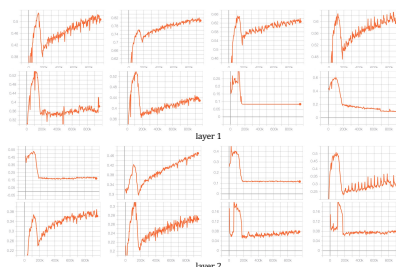


图 4: RankMixer 中不同 token 的激活专家比例。

未来 10 B 规模部署、同时不突破成本预算的可行路径。

专家均衡性与多样性。原生 Sparse MoE 经常存在专家不均衡问题, 进而导致部分专家训练不足, 最终产生“死亡专家” (几乎从不被激活的专家), 而只有少数固定专家持续被激活。图 4 表明, 将 DTISI (稠密训练、稀疏推理) 与 ReLU 路由相结合能够有效解决这一问题: 稠密训练保证大多数专家获得充分的梯度更新, 避免专家饥饿。ReLU 路由使不同 token 的激活比例动态变化——图中的激活比例会根据信息含量自适应调整, 这与推荐数据多样且高度动态的分布十分契合。

4.6 在线服务成本

当参数量增加两个数量级时, 如何避免推理延迟急剧上升? 在实际系统中, 延迟与吞吐量成反比, 并与服务机器资源成本成正比。与我们此前全面部署的 16M 参数模型 (结构融合 DLRM 和 DCN) 相比, RankMixer 将参数量扩大约 70 $\times$ 至 1B。尽管参数量显著增加, 得益于与硬件对齐的模型设计和优化策略, 最终推理延迟仍保持稳定。

在大幅增加模型参数时, 延迟可分解为以下公式:

$$\text{Latency} = \frac{\# \text{Param} \times \text{FLOPs/Param ratio}}{\text{MFU} \times (\text{Theoretical Hardware FLOPs})}$$

如表 6 所示, 两个数量级的参数增长依次被 FLOPs/Param 比率降低 3.6 $\times$ 、MFU 提高 10 $\times$, 以及基于量化的硬件 FLOPs 提升 2 $\times$ 所抵消。

表 4: RankMixer 在抖音与抖音极速版信息流推荐场景中的在线 A/B 测试结果，所有提升均具有统计显著性。根据长期反向 A/B 测试及持续观察，RankMixer-1B 带来的收益尚未收敛，本文所列结果仍在持续提升。

	抖音					抖音极速版				
	活跃天数 ↑	使用时长 ↑	点赞 ↑	完播 ↑	评论 ↑	活跃天数 ↑	使用时长 ↑	点赞 ↑	完播 ↑	评论 ↑
整体	+0.2908%	+1.0836%	+2.3852%	+1.9874%	+0.7886%	+0.1968%	+0.9869%	+1.1318%	+2.0744%	+1.1338%
低活跃	+1.7412%	+3.6434%	+8.1641%	+4.5393%	+2.9368%	+0.5389%	+2.1831%	+4.4486%	+3.3958%	+0.9595%
中活跃	+0.7081%	+1.5269%	+2.5823%	+2.5062%	+1.2266%	+0.4235%	+1.9011%	+1.7491%	+2.6568%	+0.6782%
高活跃	+0.1445%	+0.6259%	+1.828%	+1.4939%	+0.4151%	+0.0929%	+1.1356%	+1.8212%	+1.7683%	+2.3683%

表 5: RankMixer 在广告场景中的在线提升

指标	广告	
	$\Delta AUC \uparrow$	$ADV \uparrow$
提升	+0.73%	+3.90%

表 6: 在线模型部署与成本指标

指标	OnlineBase-16M	RankMixer-1B	变化
# 参数	15.8M	1.1B	$\uparrow 70\times$
FLOPs	107G	2106G	$\uparrow 20.7\times$
Flops/参数 (G/M)	6.8	1.9	$\downarrow 3.6\times$
MFU	4.47%	44.57%	$\uparrow 10\times$
硬件 FLOPs	fp32	fp16	$\uparrow 2\times$
延迟	14.5ms	14.3ms	-

- (1) **FLOPs/Param 比率**。表 6 的第三行报告了每个参数所需的浮点运算次数 (FLOPs)。得益于模型设计，RankMixer 仅以约 20 倍的 FLOPs 增幅实现了 70 倍的参数增幅，其 FLOPs/Param 比率仅为基线的三分之一，实现了 **3.6,× 倍的效率提升**。换言之，在相同的 FLOPs 预算下，RankMixer 可容纳比基线多三倍的参数。
- (2) **模型 FLOPs 利用率 (MFU)**。表 6 中也给出了 MFU，它表示机器计算能力的利用率。通过使用大尺寸 GEMM、良好的并行拓扑 (将并行的逐 token FFN 融合至一个内核)，并减少内存带宽成本和额外开销，RankMixer 将 MFU 提高了近 **10×**，使模型从内存受限转向计算受限状态。

- (3) **量化半精度 (fp16) 推理**会将 GPU 的理论硬件峰值 FLOPs 提高 **2×**。如前所述，RankMixer 的主要计算由若干个大型矩阵乘法组成，非常适合半精度 (fp16) 推理。

4.7 在线性能

为了验证 RankMixer 作为可扩展推荐模型框架的通用性，我们在个性化排序的两个核心应用场景——*Feed* 推荐和广告——中开展了在线实验，覆盖个性化排序的主要用例。对于每个场景，我们监测以下关键性能指标：

- **Feed 推荐**。活跃天数是实验期间每位用户的平均活跃天数，可作为 DAU 增长的替代指标；时长衡量用户在 App 内的累计停留时间；完播/点赞/评论：用户的完整播放、点赞和评论总数。
- **广告**。我们报告模型质量指标 (ΔAUC) 和收入指标 ADV (广告主价值)。

这些场景中此前的基线是融合 DLRM 和 DCN 的 16M 参数模型。我们使用 RankMixer-1B 模型替换其中的 Dense 部分，使 AUC 提升了 0.7%。我们在 Feed 推荐和广告场景中开展了在线 A/B 测试。Feed 推荐为期 8 个月的 A/B 测试长期观测结果见表 4¹，广告场景的结果见表 5。

¹该结果于 2025 年 7 月 24 日更新，因为长期实验表明增益尚未饱和。

RankMixer 已部署并评估于三个个性化排序应用, 其中包括 Feed 推荐(RankMixer-1B)和广告(RankMixer-1B)。RankMixer 在所有业务关键指标上均实现了具有统计显著性的提升。与其他活跃度级别的用户群体相比, 我们还可以从表 4 中观察到, 低活跃用户的提升最大, 活跃天数提升超过 1.7412%, 这说明该模型具有很强的泛化能力。这些结果表明, RankMixer 作为统一骨干网络, 能够可靠地泛化到不同应用场景。

5 结论

本文介绍了我们最新的 RankMixer 模型, 该模型已全面部署于抖音 Feed 排序。它将面向异构特征交互的模型设计与高度可并行、服务高效的架构相结合。实验表明, RankMixer 具有出色的性能和陡峭的缩放定律。在抖音 App 全量部署后, 活跃天数和 App 使用时长分别提升了 0.3% 和 1%。

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Al-tenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Newsha Ardalani, Carole-Jean Wu, Zeliang Chen, Bhargav Bhushanam, and Adnan Aziz. 2022. Understanding scaling laws for recommendation models. *arXiv preprint arXiv:2208.08489* (2022).
- [3] Fedor Borisjuk, Mingzhou Zhou, Qingquan Song, Siyu Zhu, Birjodh Tiwana, Ganesh Parameswaran, Siddharth Dangi, Lars Hertel, Qiang Charles Xiao, Xiaochen Hou, Yunbo Ouyang, Aman Gupta, Sheallika Singh, Dan Liu, Hailing Cheng, Lei Le, Jonathan Hung, S. Sathya Keerthi, Ruoyan Wang, Fengyu Zhang, Mohit Kothari, Chen Zhu, Daqi Sun, Yun Dai, Xun Luan, Sirou Zhu, Zhiwei Wang, Neil Daftary, Qianqi Shen, Chengming Jiang, Haichao Wei, Maneesh Varshney, Amol Ghoting, and Souvik Ghosh. 2024. LiRank: Industrial Large Scale Ranking Models at LinkedIn. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*. 4804–4815.
- [4] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, Xionghang Xie, Shiru Ren, Xiang Sun, Yaocheng Tan, Peng Xu, Yuchao Zheng, and Di Wu. 2025. LONGER: Scaling Up Long Sequence Modeling in Industrial Recommenders. *arXiv:2505.04421 [cs.LR]* <https://arxiv.org/abs/2505.04421>
- [5] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems*. 7–10.
- [6] Sharad Chitlangia, Krishna Reddy Kesari, and Rajat Agarwal. 2023. Scaling generative pre-training for user ad activity sequences. (2023).
- [7] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [8] Alexey Dosovitskiy. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [9] Yan Fang, Jingtao Zhan, Qingyao Ai, Jiaxin Mao, Weihang Su, Jia Chen, and Yiqun Liu. 2024. Scaling laws for dense retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1339–1349.
- [10] Huan Gui, Ruoxi Wang, Ke Yin, Long Jin, Maciej Kula, Taibai Xu, Lichan Hong, and Ed H Chi. 2023. Hiformer: Heterogeneous Feature Interactions Learning with Transformers for Recommender Systems. *arXiv preprint arXiv:2311.05884* (2023).
- [11] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiquang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *arXiv preprint arXiv:1703.04247* (2017).
- [12] Wei Guo, Hao Wang, Luankang Zhang, Jin Yao Chin, Zhongzhou Liu, Kai Cheng, Qiushi Pan, Yi Quan Lee, Wanqi Xue, Tingjia Shen, et al. 2024. Scaling New Frontiers: Insights into Large Recommendation Models. *arXiv preprint arXiv:2412.00714* (2024).
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [14] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022).
- [15] Yanhua Huang, Hangyu Wang, Yiyun Miao, Ruiwen Xu, Lei Zhang, and Weinan Zhang. 2022. Neural statistics for click-through rate prediction. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1849–1853.
- [16] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [17] Zekun Li, Zeyu Cui, Shu Wu, Xiaoyu Zhang, and Liang Wang. 2019. Fi-gnn: Modeling feature interactions via graph neural networks for ctr prediction. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 539–548.

- [18] Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xdeepfm: Combining explicit and implicit feature interactions for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1754–1763.
- [19] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, et al. 2023. How can recommender systems benefit from large language models: A survey. *ACM Transactions on Information Systems* (2023).
- [20] Bin Liu, Ruiming Tang, Yingzhi Chen, Jinkai Yu, Huifeng Guo, and Yuzhou Zhang. 2019. Feature generation by convolutional neural network for click-through rate prediction. In *The World Wide Web Conference*. 1119–1129.
- [21] Junwei Pan, Wei Xue, Ximei Wang, Haibin Yu, Xun Liu, Shijie Quan, Xueming Qiu, Dapeng Liu, Lei Xiao, and Jie Jiang. 2024. Ads recommendation in a collapsed and entangled world. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5566–5577.
- [22] Yanru Qu, Bohui Fang, Weinan Zhang, Ruiming Tang, Minzhe Niu, Huifeng Guo, Yong Yu, and Xiuqiang He. 2018. Product-based neural networks for user response prediction over multi-field categorical data. *ACM Transactions on Information Systems (TOIS)* 37, 1 (2018), 1–35.
- [23] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- [24] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. 2021. Zero-shot text-to-image generation. In *International conference on machine learning*. Pmlr, 8821–8831.
- [25] Steffen Rendle, Walid Krichene, Li Zhang, and John Anderson. 2020. Neural collaborative filtering vs. matrix factorization revisited. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 240–248.
- [26] Ying Shan, T Ryan Hoens, Jian Jiao, Haijing Wang, Dong Yu, and JC Mao. 2016. Deep crossing: Web-scale modeling without manually crafted combinatorial features. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 255–262.
- [27] Kyuyong Shin, Hanock Kwak, Su Young Kim, Max Nihlén Ramström, Jisu Jeong, Jung-Woo Ha, and Kyung-Min Kim. 2023. Scaling law for recommendation models: Towards general-purpose user representations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 4596–4604.
- [28] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. AutoInt: Automatic feature interaction learning via self-attentive neural networks. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1161–1170.
- [29] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295* (2024).
- [30] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [31] Hangyu Wang, Jianghao Lin, Xiangyang Li, Bo Chen, Chenxu Zhu, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. FLIP: Fine-grained Alignment between ID-based Models and Pre-trained Language Models for CTR Prediction. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 94–104.
- [32] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*. 1–7.
- [33] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the web conference 2021*. 1785–1797.
- [34] Yunli Wang, Zixuan Yang, Zhen Zhang, Zhiqiang Wang, Jian Yang, Shiyang Wen, Peng Jiang, and Kun Gai. 2024. Scaling Laws for Online Advertisement Retrieval. *arXiv preprint arXiv:2411.13322* (2024).
- [35] Ziteng Wang, Jianfei Chen, and Jun Zhu. 2024. ReMoE: Fully Differentiable Mixture-of-Experts with ReLU Routing. *CoRR* abs/2412.14711 (2024). doi:10.48550/ARXIV.2412.14711 arXiv:2412.14711
- [36] Jiaqi Zhang, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, et al. [n. d.]. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *Forty-first International Conference on Machine Learning*.
- [37] Xiaohua Zhai, Alexander Kolesnikov, Neil Houlsby, and Lucas Beyer. 2022. Scaling vision transformers. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 12104–12113.
- [38] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Shen Li, Yanli Zhao, Yuchen Hao, Yantao Yao, Ellie Dingqiao Wen, et al. [n. d.]. Wukong: Towards a Scaling Law for Large-Scale Recommendation. In *Forty-first International Conference on Machine Learning*.
- [39] Buyun Zhang, Liang Luo, Xi Liu, Jay Li, Zeliang Chen, Weilin Zhang, Xiaohan Wei, Yuchen Hao, Michael Tsang, Wenjun Wang, et al. 2022. DHEN: A deep and hierarchical ensemble network for large-scale click-through rate prediction. *arXiv preprint arXiv:2203.11014* (2022).
- [40] Gaowei Zhang, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, and Ji-Rong Wen. 2024. Scaling law of large sequential recommendation models. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 444–453.

- [41] Weinan Zhang, Jiarui Qin, Wei Guo, Ruiming Tang, and Xiuqiang He. 2021. Deep learning for click-through rate estimation. *arXiv preprint arXiv:2104.10584* (2021).
- [42] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (London, United Kingdom) (*KDD '18*). Association for Computing Machinery, New York, NY, USA, 1059–1068. doi:10.1145/3219819.3219823